

PROGRAMMING IN HASKELL

1



Topic 4.2 The Lambda Calculus

Lambda Calculus

- ❑ **lambda calculus** is a model of computation devised in the 1930s by Alonzo Church.
- ❑ Functional programming languages are all based on the lambda calculus.
- ❑ Haskell is a **pure** functional language because all its features are translatable into lambda expressions.
- ❑ Allows a higher degree of abstraction and composability.

The structure of lambda terms

3

Lambda abstraction

$\lambda x .$

$x + 1$

Body (expression)

uses x

Bound variable (parameter)

Lambda symbol (introduces a function)

Alpha Equivalence

We have seen the function

$$\lambda x.x$$

The variable x here is not semantically meaningful except in its role in that single expression. Because of this, there's a form of equivalence between lambda terms called alpha equivalence. In other words,

$$\lambda x.x$$
$$\lambda \text{apple}.\text{apple}$$
$$\lambda \text{orange}.\text{orange}$$

all mean the same thing.

Beta Reduction

When we apply a function to an argument, we

- substitute the input expression for all instances of bound variables within the body of the abstraction
- eliminate the head of the abstraction (its only purpose was to bind a variable)

This process is called beta reduction.

Beta Reduction - Example 1

$$\lambda x. x$$

- ❑ We apply the function above to 2
- ❑ substitute 2 for each bound variable in the body of the function, and then
- ❑ eliminate the head:

$$(\lambda x. x) \ 2$$
$$2$$

- ❑ The only bound variable is the single x , so applying this function to 2 returns 2.
- ❑ This function is the **identity** function.

Beta Reduction - Example 2

$(\lambda x. x+1)$

What happens if we apply this to 2
(try this yourself)

Beta Reduction- Example 3

- We can also apply our identity function to another lambda abstraction:

$$(\lambda x.x)(\lambda y.y)$$

- In this case, we substitute the entire abstraction in for x .
- We use a new syntax here, $[x := z]$, to indicate that z will be substituted for all occurrences of x

(here z is the function $(\lambda y.y)$)

- We reduce this application like this:

$$\begin{aligned} &(\lambda x.x)(\lambda y.y) \\ &[x := (\lambda y.y)] \\ &(\lambda y.y) \end{aligned}$$

Beta Reduction- Example 4

Once more, but this time we'll add another argument:

$$(\lambda x.x)(\lambda y.y)z$$

Applications in the lambda calculus are left associative. That is, unless specific parentheses suggest otherwise, they associate, or group, to the left. So, it can be rewritten as:

$$((\lambda x.x)(\lambda y.y))z$$

The β -reduction is as follows:

$$((\lambda x.x)(\lambda y.y))z \quad [x := (\lambda y.y)]$$

$$(\lambda y.y)z. \quad [y := z]$$

$$z$$

We can't reduce this any further as there is nothing left to apply, and we know nothing about z

Variables

Variables can be

- ☐ bound or
- ☐ free

as the λ -calculus assumes an infinite universe of free variables. They are bound to functions in an environment, then they become bound by usage in an abstraction.

For example, in the λ -expression:

$$(\lambda x. x * y)$$

x is bound by λ over the body $x * y$, but y is a free variable. When we apply this function to an argument, nothing can be done with the y . It remains irreducible.

Variables contd.

Look at the following when we apply such a function to an argument:

$$(\lambda x. x * y)z$$

We apply the lambda to the argument z.

$$\begin{array}{l} (\lambda x. x * y)z \\ [x := z] \\ zy \end{array}$$

The head has been applied away, and there are no more heads or bound variables. Since we know nothing about z or y, we can reduce this no further.

Multiple Arguments

- ❑ Each lambda can only bind one parameter and can only accept one argument. Functions that require multiple arguments have multiple, nested heads. When you apply it once and eliminate the first (leftmost) head, the next one is applied and so on. This means that the following

$$\lambda xy.xy$$

- ❑ is simply syntactic sugar for

$$\lambda x (\lambda y.xy)$$

Haskell version. `\x y -> x*y`

if expression and lambdas

- In Haskell, **if is an expression**, not a statement.
- That means it can appear **anywhere an expression is allowed** — including inside a lambda.

if expression and lambdas

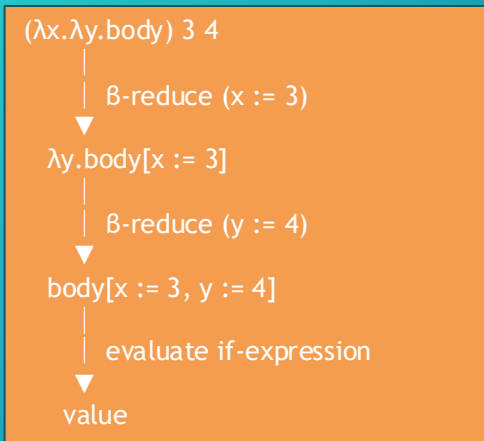
```
\x -> \y ->  
  if x < y then -1  
  else 1
```

Definition of
function

Application of
function

```
(\x y -> if x < y then -1 else 1) 3 4  
-1
```

if expression and lambdas



Lambda Calculus in Haskell

How do we write lambda expressions in Haskell?

Named Function	Lambda Calculus (maths)	Lambda Calculus (Haskell)	Result
$f\ x = x + 1$	$(\lambda x. x + 1)\ 2$	$(\backslash x \rightarrow x + 1)\ 2$	3
$f\ x\ y = x * y$	$(\lambda x\ y. x * y)\ 2\ 3$	$(\backslash x\ y \rightarrow x * y)\ 2\ 3$	6
$f\ xs = 'c' : xs$	$(\lambda xs. 'c' : xs)\ \text{"at"}$	$(\backslash xs \rightarrow 'c' : xs)\ \text{"at"}$	"cat"

Lambda functions are used extensively in Haskell, notably with Higher Order Functions.

Possible Errors - 1 - missing else

```
\x -> if x > 0 then x
```



What's wrong?

if must return a value in *all* cases - always use an else.

```
\x -> if x > 0 then x else 0
```



Possible Errors - 2 - Applying arguments to the wrong thing

```
\x y -> if x < y then -1 else 1 3 4
```



What's wrong?

3 and 4 are being applied to 1, not to the lambda.

```
(\x y -> if x < y then -1 else 1) 3 4
```



Possible Errors - 3 - Forgetting lambdas bind one variable

`\xy -> x + y`



What's wrong?
xy is a single variable name

`\x y -> x + y`



Possible Errors - 4 - Trying to evaluate before application

```
\y -> if x < y then -1 else 1
```



What's wrong?

x is not bound anywhere

```
\x y -> if x < y then -1 else 1
```





**ANY
QUESTIONS?**